PES-VCS: Local Version Control System Report

Name: Sujal Sachin Yadavi
SRN: PES1UG24CS475
Repository: https://github.com/Aqual456/PES1UG24CS475-pes-vcs
Platform: Kali Linux (Ubuntu-based)

Table of Contents

1. Setup
2. Phase 1: Object Store
3. Phase 2: Tree Objects
4. Phase 3: Staging Area
5. Phase 4: Commits
6. Phase 5: Written Questions
7. Integration Test

Setup

Prerequisites Installed

```
sudo apt update && sudo apt install -y gcc build-essential libssl-dev
```

Repository Cloned

```
git clone https://github.com/Aqual456/PES1UG24CS475-pes-vcs.git
cd PES1UG24CS475-pes-vcs
export PES_AUTHOR="Sujal_Sachin_Yadavi <PESXUG24CS475>"
```

Phase 1: Object Store (`object.c`)

Description

Implemented a content-addressable storage system where every file is stored by its SHA-256 hash. Same content = same hash = stored only once (deduplication).

Functions Implemented

`object_write` — Store data in the object store

- Builds header string (`"type size\0"`)
- Concatenates header + data into full object

- Computes SHA-256 hash of the full object
- Checks for deduplication (skips if object already exists)
- Creates shard directory (`.pes/objects/XX/`)
- Writes to temporary file with `fsync()` for durability
- Atomic rename from temp to final path

`object_read` — Read and verify data from the object store

- Builds file path from hash
- Reads entire file into memory
- Verifies integrity by recomputing SHA-256 and comparing
- Parses header to extract type (blob/tree/commit)
- Returns data portion after the null terminator

Test Results

Screenshot 1A: Output of `./test_objects` showing all tests passing

```
┌──(kali㉿kali)-[~/Desktop/PES1UG24CS475-pes-vcs]
└─$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 1B: Sharded directory structure (`find .pes/objects -type f`)

```
┌──(kali㉿kali)-[~/Desktop/PES1UG24CS475-pes-vcs]
└─$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
```

Phase 2: Tree Objects (`tree.c`)

Description

Implemented tree objects that represent directory snapshots. Each tree entry points to a blob (file) or another tree (subdirectory), enabling hierarchical storage.

Functions Implemented

`tree_from_index` — Build tree hierarchy from staged files

- Sorts index entries by path for correct grouping

- Recursively builds subtrees for nested directories
- Files at current level are added as blob entries
- Subdirectories are recursively processed into tree objects
- Serializes and writes tree objects to the object store

Helper Functions

- `compare_entries_by_path` — Sort comparator for index entries
- `build_tree_recursive` — Recursive tree construction from flat path list

Test Results

Screenshot 2A: Output of `./test_tree` showing all tests passing

```
┌──(kali㉿kali)-[~/Desktop/PES1UG24CS475-pes-vcs]
└─$ make test_tree
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -o test_tree test_tree.o object.o tree.o index.o -lcrypto

┌──(kali㉿kali)-[~/Desktop/PES1UG24CS475-pes-vcs]
└─$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2B: Raw binary format of a tree object (xxd)

```
┌──(kali㉿kali)-[~/Desktop/PES1UG24CS475-pes-vcs]
└─$ find .pes/objects -type f
.pes/objects/e2/8b7551fbff6cf3eb5c2efc0503580965f532823139790afcd8b89caf112952
```

```
┌──(kali㉿kali)-[~/Desktop/PES1UG24CS475-pes-vcs]
└─$ xxd .pes/objects/e2/8b7551fbff6cf3eb5c2efc0503580965f532823139790afcd8b89caf112952 | head -20
00000000: 7472 6565 2034 3900 3130 3036 3434 2066  tree 49.100644 f
00000010: 696c 6531 2e74 7874 00ab abab abab abab  ile1.txt........
00000020: abab abab abab abab abab abab abab abab  ................
00000030: abab abab abab abab ab                  .........
```

Phase 3: Staging Area (`index.c`)

Description

Implemented the staging area (index) that tracks which files are ready to be committed. The index is stored as a text file at `.pes/index`.

Functions Implemented

`index_load` — Read the index file into memory

- Opens `.pes/index` for reading

- Parses each line: `<mode> <hash> <mtime> <size> <path>`
- Handles missing index file gracefully (empty index for first use)

`index_save` — Write the index atomically

- Sorts entries by path using `qsort()`
- Writes to temporary file `.pes/index.tmp`
- Flushes with `fflush()` and `fsync()` for durability
- Atomic rename from temp to final index file

`index_add` — Stage a file for commit

- Reads file contents from working directory
- Creates blob object via `object_write()`
- Gets file metadata via `stat()` (mode, mtime, size)
- Updates existing entry or adds new entry to index
- Saves updated index to disk

Test Results

Screenshot 3A: Output of `pes init` + `pes add` + `pes status`

```
┌──(kali㉿kali)-[~/Desktop/PES1UG24CS475-pes-vcs]
└─$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

```
┌──(kali㉿kali)-[~/os-u4-orange-problem]
└─$ rm -rf .pes
./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status

Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  pes.c
  untracked:  .gitignore
  untracked:  tree.h
  untracked:  test_tree.c
  untracked:  index.h
  untracked:  tree.c
  untracked:  commit.h
  untracked:  README.md
  untracked:  object.c
  untracked:  index.c
  untracked:  test_objects.c
  untracked:  test_sequence.sh
  untracked:  commit.c
  untracked:  hello.txt
  untracked:  pes.h
  untracked:  Makefile
  untracked:  test_tree
```

Screenshot 3B: Contents of `.pes/index` showing text-format entries

```
┌──(kali㉿kali)-[~/os-u4-orange-problem]
└─$ cat .pes/index

100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776253890 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776253890 6 file2.txt
```

Phase 4: Commits (`commit.c`)

Description

Implemented the commit system that ties everything together — commits point to a tree (project snapshot) and contain metadata (author, timestamp, message, parent commit).

Functions Implemented

`commit_create` — Create a new commit

- Builds tree from current index via `tree_from_index()`
- Reads current HEAD as parent (handles first commit with no parent)
- Sets author from `PES_AUTHOR` environment variable
- Sets timestamp to current Unix time
- Serializes commit to text format
- Writes commit object to the object store
- Updates HEAD to point to the new commit

Test Results

Screenshot 4A: `pes log` showing commit history with multiple commits

```
┌──(kali㉿kali)-[~/os-u4-orange-problem]
└─$ ./pes commit -m "First commit"
echo "more stuff" > file3.txt
./pes add file3.txt
./pes commit -m "Add file3"
echo "updated" >> file1.txt
./pes add file1.txt
./pes commit -m "Update file1"
./pes log

Committed: d0220fb07627 ... First commit
Committed: 2c6379c4f0bc ... Add file3
Committed: 94562ccdb9e0 ... Update file1
commit 94562ccdb9e0f6376cd4cb8dd91569fcdce28002b620b5802c57217c6ec23e50
Author: Faizan  <PESXUG24AM472>
Date:   1776253996

    Update file1

commit 2c6379c4f0bc137e0a92f3039970c600f460ebaa102f63ae9a17ecf5ba8880d2
Author: Faizan  <PESXUG24AM472>
Date:   1776253996

    Add file3

commit d0220fb07627619d8d3f61a57ca32a7b05084f8761d05a9ffc16393ed61ef9ac
Author: Faizan  <PESXUG24AM472>
Date:   1776253996

    First commit
```

Screenshot 4B: Object store growth (`find .pes -type f | sort`)

```
┌──(kali㉿kali)-[~/os-u4-orange-problem]
└─$ find .pes -type f | sort

.pes/HEAD
.pes/index
.pes/objects/24/12cce4b97c09ebb1b604884ca66f141c2364a05858fef8ac4859da40474bec
.pes/objects/2c/6379c4f0bc137e0a92f3039970c600f460ebaa102f63ae9a17ecf5ba8880d2
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/64/5b507be548973ff35eab64c5e6efb7dd5e7d419484fafe519f60dd4b88d889
.pes/objects/94/562ccdb9e0f6376cd4cb8dd91569fcdce28002b620b5802c57217c6ec23e50
.pes/objects/d0/220fb07627619d8d3f61a57ca32a7b05084f8761d05a9ffc16393ed61ef9ac
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e9/37ea375e05180cda902480ca6a1e073c6182426466e05bf019b997ce1c2324
.pes/objects/f5/902ee73b6aa25436f687067fc4cc0cb481cf416740d3c3c3ed6d9c4de7b6c2
.pes/objects/ff/0067fdedc7a8b2eeae738b45e3b32720f095f72de7eca3e3dabc5af796ddce
.pes/refs/heads/main
```

Screenshot 4C: Reference chain (`cat .pes/refs/heads/main` and `cat .pes/HEAD`)

```
┌──(kali㉿kali)-[~/os-u4-orange-problem]
└─$ cat .pes/HEAD
cat .pes/refs/heads/main

ref: refs/heads/main
94562ccdb9e0f6376cd4cb8dd91569fcdce28002b620b5802c57217c6ec23e50
```

Integration Test

Screenshot 4D: Integration test output

```
┌──(kali㉿kali)-[~/os-u4-orange-problem]
└─$ make test-integration

═══ Running integration tests ═══
bash test_sequence.sh
═══ PES-VCS Integration Test ═══

── Repository Initialization ──
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

── Staging Files ──
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


── First Commit ──
Committed: 2eec8d21c54b... Initial commit

Log after first commit:
commit 2eec8d21c54bfc5f8220a5d08e0333757a455de3ea32ff58c6ee7641a2ccacf2
Author: Faizan  <PESXUG24AM472>
Date:   1776254176

    Initial commit


── Second Commit ──
Committed: 680c56b6840b... Update file.txt

── Third Commit ──
Committed: 357360bfcfb6... Add farewell

── Full History ──
commit 357360bfcfb631d98d598aa06d0e6ee16cf0806bfe2a77108c79a46cdbc9179c
```

```
── Full History ──
commit 357360bfcfb631d98d598aa06d0e6ee16cf0806bfe2a77108c79a46cdbc9179c
Author: Faizan  <PESXUG24AM472>
Date:   1776254176

    Add farewell

commit 680c56b6840b98b631463d393aa6db4afc08840818242bf9e663790cdd9e62be
Author: Faizan  <PESXUG24AM472>
Date:   1776254176

    Update file.txt

commit 2eec8d21c54bfc5f8220a5d08e0333757a455de3ea32ff58c6ee7641a2ccacf2
Author: Faizan  <PESXUG24AM472>
Date:   1776254176

    Initial commit


── Reference Chain ──
HEAD:
ref: refs/heads/main
refs/heads/main:
357360bfcfb631d98d598aa06d0e6ee16cf0806bfe2a77108c79a46cdbc9179c

── Object Store ──
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/2e/ec8d21c54bfc5f8220a5d08e0333757a455de3ea32ff58c6ee7641a2ccacf2
.pes/objects/35/7360bfcfb631d98d598aa06d0e6ee16cf0806bfe2a77108c79a46cdbc9179c
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/68/0c56b6840b98b631463d393aa6db4afc08840818242bf9e663790cdd9e62be
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

═══ All integration tests completed ═══
```

Phase 5: Written Questions

Q5.1: How would you implement `pes checkout <branch>`?

To checkout a branch:

1. **Read the target branch** file from `.pes/refs/heads/<branch>` to get its commit hash.
2. **Update `.pes/HEAD`** to `ref: refs/heads/<branch>`.
3. **Read the commit object** → get the tree hash → recursively read the tree.
4. **Update the working directory** files to match the tree snapshot.
5. **Update the index** to match the new tree.

Complexity: The hard part is handling conflicts — if the user has uncommitted changes to files that differ between branches, checkout must refuse. You'd also need to handle file creation/deletion when switching between trees with different file sets.

Q5.2: How to detect uncommitted changes using only the index and object store?

1. For each entry in the index, `stat()` the working directory file.
2. Compare `mtime` and `size` from `stat` with the index entry's stored values.
3. If metadata differs, re-hash the file contents and compare with the stored blob hash.
4. If the hash differs, the file has uncommitted changes → refuse checkout.
5. Also check for deleted files (`stat` fails) and new untracked files.

Q5.3: What happens with commits in detached HEAD state?

In detached HEAD, `HEAD` contains a commit hash directly (not `ref: refs/heads/...`). New commits still work — each new commit's hash replaces the old hash in HEAD.

However, if you then checkout a branch, the detached commits become **"orphaned"** — no branch points to them. They're unreachable but still exist in the object store.

To recover: Note the commit hash before switching, then create a branch pointing to it:

```
git branch rescue-branch <hash>
```

Q6.1: Describe an algorithm to find and delete unreachable objects (Garbage Collection)

1. **Mark phase:** Start from all branch tips (files in `refs/heads/`). For each reachable commit, mark it and follow: commit → tree → recursively mark all subtrees and blobs. Use a **hash set** to track reachable hashes.

2. **Sweep phase:** Walk all objects in `.pes/objects/`. Delete any object whose hash is NOT in the reachable set.

3. **Estimation:** For 100K commits with 50 branches, you'd visit ~100K commits + their trees. If each commit has ~100 files, that's ~10M tree/blob references to check (but deduplication means far fewer unique objects).

Q6.2: Why is concurrent GC + commit dangerous?

Race condition: A commit operation creates objects in stages — first blobs, then trees, then the commit object, then updates the ref. If GC runs between "write tree" and "update ref," it could see the new tree as unreachable (no ref points to it yet) and delete it. When the commit operation tries to update the ref, it points to a deleted object.

Git's solution: Git uses a lock file (`.git/gc.lock`) and a grace period — GC only deletes objects older than **2 weeks** by default (`gc.pruneExpire`). This ensures in-progress operations complete before their objects become eligible for deletion.

Git Commit History

Phase 1 Commits

```
Phase 1: Implement object_write header construction
Phase 1: Add SHA-256 hashing and deduplication check
Phase 1: Add atomic write with fsync and rename
Phase 1: Implement object_read with file parsing
Phase 1: Add integrity verification in object_read
```

Phase 2 Commits

```
Phase 2: Add tree entry sorting helper
Phase 2: Implement recursive tree builder
Phase 2: Implement tree_from_index with index loading
Phase 2: Handle nested directory paths in tree builder
Phase 2: Add error handling for tree serialization
```

Phase 3 Commits

```
Phase 3: Implement index_load text parser
Phase 3: Implement index_save with atomic write
Phase 3: Implement index_add with blob creation
Phase 3: Add file metadata tracking (mtime, size)
Phase 3: Add index entry update for re-staged files
```

Phase 4 Commits

```
Phase 4: Implement commit_create with tree building
Phase 4: Add parent commit linking
Phase 4: Add author and timestamp metadata
Phase 4: Connect commit to HEAD update
Phase 4: Add error handling for commit workflow
```

Phase 5 Commits

```
Phase 5: Add written answers for design questions
```

End of Report